

Permutation Invariance and Equivariance in GNNs

A matrix-based classroom tutorial with a worked example and Python code

Permutation Invariance: $f(P A P^T) = f(A)$ - The function output is unchanged by node relabeling

Permutation Equivariance: $f(P A P^T) = P f(A)$ - The function output is permuted in the same way as the input

Learning objectives

- Interpret an adjacency matrix and identify graph connections.
- Construct a permutation matrix for a selected node reordering.
- Calculate the permuted adjacency matrix $A' = P A P^T$.
- Distinguish graph-level invariance from node-level equivariance.
- Verify both properties using NumPy.

1. Original graph and adjacency matrix

Consider an undirected graph containing four nodes in the order [0, 1, 2, 3]. Its adjacency matrix is:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

An entry $A_{ij} = 1$ means that nodes i and j are connected. Because A is symmetric, the graph is undirected.

Edge	Interpretation
(0, 1)	Node 0 is connected to node 1
(0, 2)	Node 0 is connected to node 2
(1, 2)	Node 1 is connected to node 2
(1, 3)	Node 1 is connected to node 3

2. Calculate the node degrees

The degree of node i is the sum of row i :

Node	Row sum	Degree
0	$0 + 1 + 1 + 0$	2
1	$1 + 0 + 1 + 1$	3
2	$1 + 1 + 0 + 0$	2
3	$0 + 1 + 0 + 0$	1

$$d(A) = [2, 3, 2, 1]$$

3. Swap nodes 1 and 3

Change only the order in which the nodes are written:

$$[0, 1, 2, 3] \rightarrow [0, 3, 2, 1]$$

No edge is added or removed. Node 3 moves to the second position and node 1 moves to the fourth position.

4. Construct the permutation matrix

Start with the 4×4 identity matrix and swap rows 1 and 3:

$$P = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

New position	Original node selected	Row of P
0	0	[1, 0, 0, 0]
1	3	[0, 0, 0, 1]
2	2	[0, 0, 1, 0]
3	1	[0, 1, 0, 0]

Rule: to swap nodes i and j , start with I_n and swap rows i and j . A valid permutation matrix has exactly one 1 in every row and every column.

5. Calculate $A' = PAP^T$

Step 1: Left multiplication by P

PA rearranges the rows of A into the order [0, 3, 2, 1]:

$$PA = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 \end{bmatrix}$$

Step 2: Right multiplication by P^T

P^T rearranges the columns into the same order:

$$A' = PAP^T = \begin{bmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

The row and column labels of A' are [0, 3, 2, 1], not [0, 1, 2, 3].

Row in A'	Node	Meaning
[0, 0, 1, 1]	Original 0	Connected to original nodes 2 and 1
[0, 0, 0, 1]	Original 3	Connected only to original node 1
[1, 0, 0, 1]	Original 2	Connected to original nodes 0 and 1

[1, 1, 1, 0]	Original 1	Connected to original nodes 0, 3 and 2
--------------	------------	--

A and A' look different, but their connections are identical. Only the positions of the node labels have changed.

6. Permutation invariance: graph-level output

A graph-level function gives one value for the whole graph. It is permutation invariant when:

$$g(PAP^T) = g(A)$$

Let g count the number of nodes. Both matrices represent a graph with four nodes:

Input	Output
A	$g(A) = 4$
$A' = PAP^T$	$g(A') = 4$

The value 4 comes from the definition of this example function. It counts nodes. It is not the sum of the entries of A; that sum is 8 because every undirected edge is recorded twice.

Chemistry example

A molecular GNN may predict total energy, solubility, toxicity or band gap. Such a graph-level property must not change when the atoms are listed in a different order.

7. Permutation equivariance: node-level output

A node-level function gives one output for every node. It is permutation equivariant when:

$$f(PAP^T) = P f(A)$$

Using node degree as the node-level function:

Quantity	Vector	Node order
$f(A)$	[2, 3, 2, 1]	[0, 1, 2, 3]
$f(PAP^T)$	[2, 1, 2, 3]	[0, 3, 2, 1]
$P f(A)$	[2, 1, 2, 3]	[0, 3, 2, 1]

Degree 1 moves with original node 3 to the second position. Degree 3 moves with original node 1 to the fourth position. The values follow their nodes.

Chemistry example

If a GNN predicts one partial charge for every atom, reordering the atoms must reorder the charge vector in the same way. Each charge must remain attached to its atom.

8. Invariance versus equivariance

Property	Output	Condition	Meaning
Invariance	One graph value	$g(PAP^T) = g(A)$	The value stays
Equivariance	One value per node	$f(PAP^T) = P f(A)$	The values move with nodes

Invariant: the answer stays. **Equivariant:** the answers move.

9. Complete Python demonstration

```
import numpy as np

A = np.array([
    [0, 1, 1, 0],
    [1, 0, 1, 1],
    [1, 1, 0, 0],
    [0, 1, 0, 0]
])

P = np.eye(4, dtype=int)
P[[1, 3]] = P[[3, 1]]

A_permuted = P @ A @ P.T

print("A = ", A)
print("P = ", P)
print("PAP^T = ", A_permuted)
```

Test graph-level invariance

```
def graph_level_function(A):
    return A.shape[0]    # number of nodes
```

```
print(graph_level_function(A))
print(graph_level_function(A_permuted))
assert graph_level_function(A) == graph_level_function(A_permuted)
```

Test node-level equivariance

```
def node_level_function(A):
    return A.sum(axis=1) # node degrees

f_A = node_level_function(A)
f_A_permuted = node_level_function(A_permuted)
P_f_A = P @ f_A

print("f(A)      =", f_A)
print("f(PAP^T) =", f_A_permuted)
print("P f(A)    =", P_f_A)
assert np.array_equal(f_A_permuted, P_f_A)
```

10. Classroom exercise

Using the same adjacency matrix, swap nodes 0 and 2.

1. Write the new node ordering.
2. Construct the permutation matrix P .
3. Calculate $A' = PAP^T$.
4. Calculate the original and permuted degree vectors.
5. Verify $f(PAP^T) = Pf(A)$.

Student starter code

```
import numpy as np

A = np.array([
    [0, 1, 1, 0],
    [1, 0, 1, 1],
    [1, 1, 0, 0],
    [0, 1, 0, 0]
])

# TODO 1: Create the identity matrix
P = _____

# TODO 2: Swap rows 0 and 2
P[[0, 2]] = _____

# TODO 3: Calculate  $PAP^T$ 
A_permuted = _____

# TODO 4: Calculate the degree vectors
degree_original = _____
degree_permuted = _____

# TODO 5: Test equivariance
expected_degree = _____
is_equivariant = _____
```